

WEEK 1:

REC-Digital Cafe

Question 1
Correct
Marked out of 1.00
Flag question

The code given below contains instructions to print the text **"I love Apples"** to the console.

The `\n` in the text `"I love Apples\n"` ensures that the line breaks after printing the text `"I love Apples"` (which means that nothing else is printed on the same line).

Follow the steps given below to change the text, execute **compile** command and finally **execute** the file :

1. In the code given below, change the text to print **"I love Mangoes"** instead of **"I love Apples"**.

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("I love Mangoes\n");
6     return 0;
7 }
```

Expected	Got
✓ I love Mangoes	I love Mangoes ✓

Passed all tests! ✓

➔ Thus, the program is executed successfully...

REC-Digital Cafe

Question 2
Correct
Marked out of 1.00
Flag question

Given below is a simple program written in **C** language.

Change the text in the code given below to make the program print **"Hello C"** instead of **"Hello B"**.

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("Hello C");
6     return 0;
7 }
```

Expected	Got
✓ Hello C	Hello C ✓

Passed all tests! ✓

➔ Thus, the program is executed successfully...

REC Digital Cafe

Question 3
Correct
Marked out of 100
Flag question

A non-executable computer program is a collection of instructions or statements.
A program usually consists of multiple statements.
Each statement is composed of one or more of the items given below:
1. Comments
2. Whitespace characters
3. Tokens

In a computer program, a comment is used to mark a section of code as non-executable.
Comments are mainly used for two purposes:
1. To mark a section of non-executable code as non-executable, so that the compiler ignores it during compilation.
2. To provide remarks or an explanation on the working of the given section of code in plain English, so that a fellow programmer can read and understand the code.

NOTE: There are two types of comments:
1. **single-line comment**: It starts with // The content that follows the // and continues till the end of that line is a comment. It is also called as **single-line comment**.
2. **multi-line comment**: It starts with /* and ends with */. The content between /* and */ is the comment. It is also called as **multi-line comment**.

The code given below shows the two types of comments:

```
// 1 programing language was developed by Dennis Ritchie.
// This is called a single-line comment which is used to describe
// what this program would do. As you can notice the comment is
// spanning across multiple lines.

/* Another multi-line
comment */

int main()
{
    get_int() > 30, add > 30
    printf("Sum of two numbers is %d", sum > 30);
    return 0;
}
// End of the main() function - this is an example of a multi-line comment
```

Read the code given below to understand the different types of comments. Pay attention to the space provided.

Given below are 3 important points regarding comments:

- There should not be any space between the two forward slashes (//) or /* and */. Instead, there should not be any space between the **dash** and **star** characters in /* and */. /* and */ are reserved.
- Comments do not end**: In /* and */ comment has no special meaning inside of comments. Similarly, // comment has no special meaning inside of comments.
- One should not write comments inside double quotes (i.e., characters enclosed between single quotes). Comments inside String literals (i.e., text enclosed between double quotes) are treated as part of the String's content.

Consider the below program:

```
// This is a sample C program
// developed by REC
//
// Module - module1
//
// get_int()
//
// This is an example of a comment
printf("Sum C Language")
return 0;
}
```

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("Dennis Ritchie\nBrian Kernighan");
6     return 0;
7 }
```

Expected	Got
✓ Dennis Ritchie Brian Kernighan	Dennis Ritchie Brian Kernighan ✓

Passed all tests! ✓

➔ Thus, the program is executed successfully...

REC-Digital Cafe

Question 4
Correct
Marked out of 100
Flag question

The code given below contains text that prints "DennisRitchieBrianKernighan".
Make the suggested changes to the code so that it prints "DennisRitchieBrianKernighan" as shown below.
Dennis Ritchie
Brian Kernighan

To make the required changes, follow the steps given below to introduce the SPACE character and the \n new line character appropriately:
1. Insert a space between "Dennis" and "Ritchie". Make sure that no extra space or any other character apart from space are inserted.
2. Insert a \n between "Ritchie" and "Brian". Make sure that no extra space or any other character apart from \n are inserted.
3. Insert a space between "Brian" and "Kernighan". Make sure that no extra space or any other character apart from space are inserted.

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("Dennis Ritchie\nBrian Kernighan");
6     return 0;
7 }
```

Expected	Got
✓ Dennis Ritchie Brian Kernighan	Dennis Ritchie Brian Kernighan ✓

Passed all tests! ✓

➔ Thus, the program is executed successfully...

Started on

Monday, 23 February 2026, 4:59 PM

Status

Finished

Completed on

Monday, 23 February 2026, 4:10 PM

Time taken

7 mins 11 secs

Name

SACHINEL C-2025-BME

Course

1

Answer

Read the code given below to learn naming conventions in identifiers.
For example, consider the program given below
#include <stdio.h>

int main()
{
 int age = 10; // age is an integer variable

 int firstNumber = 2; // firstNumber is an integer variable

 // If there are two or more words in an identifier/variable, then you can use "camel case" style to declare a variable.
 // second_number is an integer variable

 // Any space cannot be used between two words of an identifier/variable, then you can use underscore (_) instead of space.
 int _is_this_a_valid_identifier = 4; // _is_this_a_valid_identifier is an integer variable

 // An identifier/variable name must be start with an alphabet or underscore (_) only, no other special characters, digits are allowed as first character of the identifier/variable name.

 printf("age = %d", age);
 printf("firstNumber = %d", firstNumber);
 printf("second number = %d", second_number);
 printf("_is_this_a_valid_identifier = %d", _is_this_a_valid_identifier);
 return 0;
}

Fill in the missing code in the below program to print the values of the given variables.

Answer: (partially legible 0/0)

Read answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int age = 10; // age is an integer variable
6     int firstNumber = 2; // firstNumber is an integer variable
7     // second_number is an integer variable
8     int _is_this_a_valid_identifier = 4; // _is_this_a_valid_identifier is an integer variable
9     printf("age = %d", age); // fill in the missing code
10    printf("firstNumber = %d", firstNumber); // fill in the missing code
11    printf("second number = %d", second_number); // fill in the missing code
12    printf("_is_this_a_valid_identifier = %d", _is_this_a_valid_identifier); // fill in the missing code
13    return 0;
14 }
```

Expected	Got
age = 10	age = 2
firstNumber = 2	firstNumber = 2
second number = 3	second number = 3
_is_this_a_valid_identifier = 4	_is_this_a_valid_identifier = 4

Passed all tests! ✓

➔ Thus, the program is executed successfully...

REC Digital Cafe

CS23232-Fundamentals of Data Structures Using C-2025 Batch

Started on

Tuesday, 24 February 2026, 10:58 AM

Status

Finished

Completed on

Tuesday, 24 February 2026, 11:01 AM

Time taken

3 mins 3 secs

Name

SACHINEL C-2025-BME

Course

1

Answer

We shall learn more about functions in the later sections. For now, we can safely assume that **function** is the name given to a set of one or more executable statements. **main()** is a **user-defined function**. i.e., a user (or programmer) writes the code for the **main()** function.
While executing a C program, the **Operating System (OS)** only calls the **main()** function in that program.
When the **OS** executes a program, the program usually returns an integer value 0 if the execution of that program is successful.

In **C, main()** can be written in such a way that it returns a **int**.

#include <stdio.h>

int main()
{
 printf("Hello! Function with int as return type!!");
 return 0; // 0 value indicates that the execution is successful.
}

If the programmer does not specify any return type, the return type is by default considered as **int**.
The name of the **main()** function should always be lowercase, i.e., if a function is written as **Main()**, it is not the **main** function which is called by the **OS**.
Read the code given below to familiarize yourself with the syntax of **main()** function. Replace in the space provided.

#include <stdio.h>

int main()
{
 printf("Impossible is nothing!");
 return 0;
}

Answer: (partially legible 0/0)

Read answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("Impossible is nothing!");
6     return 0;
7 }
```

Expected	Got
impossible is nothing	impossible is nothing

Passed all tests! ✓

➔ Thus, the program is executed successfully...

Question 2
Correct
Marked out of 1.00
Flag question

Identify and correct the error in the code given below.

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("Hello, # is a preprocessor in C");
6     return 0;
7 }
```

Expected	Got
✓ Hello, # is a preprocessor in C	Hello, # is a preprocessor in C ✓

Passed all tests! ✓

➔ Thus, the program is executed successfully...

Question 3
Correct
Marked out of 1.00
Flag question

Click on **Check** without correcting the code.

This results in many errors because the main function is not defined correctly.

Now, correct the spelling of the main function and submit the program once again.

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("Correct Me!");
6     return 0;
7 }
```

Expected	Got
✓ Correct Me!	Correct Me! ✓

Passed all tests! ✓

➔ Thus, the program is executed successfully...

Question 4
Correct
Marked out of 1.00
Flag question

Identify and correct the error in the code given below.

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("Hello, float data type allocates 4 bytes in memory");
6     return 0;
7 }
```

	Expected	Got	
✓	Hello, float data type allocates 4 bytes in memory	Hello, float data type allocates 4 bytes in memory	✓

Passed all tests! ✓

➔ Thus, the program is executed successfully...

Question 5
Correct
Marked out of 1.00
Flag question

Identify and correct the error in the code given below.

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2 int main()
3 {
4     printf("Hello, I am learning C Language!");
5     return 0;
6 }
```

	Expected	Got	
✓	Hello, I am learning C Language!	Hello, I am learning C Language!	✓

Passed all tests! ✓

➔ Thus, the program is executed successfully...

Question 1
Correct
Marked out of 1.00
Flag question

Identify the error and correct the code. [Hint: Verify if all variables are declared before they are first used.]

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int number1 = 20, number2 = 30;
6     int sub;
7     sub = number1 - number2;
8     printf("The difference of the two given numbers = %d\n", sub);
9     return 0;
10 }
11
```

Expected	Got
✓ The difference of the two given numbers = -10	The difference of the two given numbers = -10 ✓

Passed all tests! ✓

➔ Thus, the program is executed successfully...

REC-Digital Cafe

Question 2
Correct
Marked out of 1.00
Flag question

In the program given below, we shall learn how to assign values to int data type from binary, octal, hex and character literals.

Read the code given below and retype in the space provided.

```
#include <stdio.h>
int main()
{
    int binaryThree = 0b11;
    printf("binaryThree value = %d\n", binaryThree);
    int octalEight = 010;
    printf("octalEight value = %d\n", octalEight);
    int hexTen = 0xA;
    printf("hexTen value = %d\n", hexTen);
    int asciiValueOfOne = '1';
    printf("asciiValueOfOne value = %d\n", asciiValueOfOne);
    int asciiValueOfA = 'A';
    printf("asciiValueOfA value = %d\n", asciiValueOfA);
    return 0;
}
```

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int binaryThree = 0b11;
6     printf("binaryThree value = %d\n", binaryThree);
7     int octalEight = 010;
8     printf("octalEight value = %d\n", octalEight);
9     int hexTen = 0xA;
10    printf("hexTen value = %d\n", hexTen);
11    int asciiValueOfOne = '1';
12    printf("asciiValueOfOne value = %d\n", asciiValueOfOne);
13    int asciiValueOfA = 'A';
14    printf("asciiValueOfA value = %d\n", asciiValueOfA);
15    return 0;
16 }
17
```

Expected	Got
✓ binaryThree value = 3 octalEight value = 8 hexTen value = 10 asciiValueOfOne value = 49 asciiValueOfA value = 65	binaryThree value = 3 octalEight value = 8 hexTen value = 10 asciiValueOfOne value = 49 asciiValueOfA value = 65 ✓

Passed all tests! ✓

➔ Thus, the program is executed successfully...

Question 3
Correct
Marked out of 1.00
Flag question

In the program given below, fill in the missing code to add two integer numbers.

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int num1 = 15, num2 = 25, sum;
6     printf("Given integers are num1 = %d, num2 = %d\n", num1, num2);
7     sum = num1 + num2;
8     printf("Sum of 2 given numbers = %d\n", sum);
9     return 0;
10 }
```

Expected	Got	
✓ Given integers are num1 = 15, num2 = 25 Sum of 2 given numbers = 40	Given integers are num1 = 15, num2 = 25 Sum of 2 given numbers = 40	✓

Passed all tests! ✓

➔ Thus, the program is executed successfully...

Question 4
Correct
Marked out of 1.00
Flag question

To print unsigned values on the console, use %u format character instead of %d in the **printf()** function.

Whenever an attempt is made to assign a negative number to an **unsigned int** (For eg: unsigned int num = -1;) the compiler does not flag it as an **error**. Instead, it will automatically convert the negative number to a positive number as shown below:
unsigned int num = -1;
The value stored in num = unsigned int maximum_value + 1 - num;
The final value in num = 4294967295 (in a 32-bit processing system)

In the program given below, fill in the missing **format characters** to print **signed** and **unsigned** values.

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     signed int number1 = -20, number2 = 20;
6     unsigned int number3 = -1, number4 = 1;
7     printf("Given signed values are %d and %d\n", number1, number2); // Fill the correct format character after %
8     printf("Given unsigned values are %u and %u\n", number3, number4); // Fill the correct format character after %
9     return 0;
10 }
```

Expected	Got	
✓ Given signed values are -20 and 20 Given unsigned values are 4294967295 and 1	Given signed values are -20 and 20 Given unsigned values are 4294967295 and 1	✓

Passed all tests! ✓

➔ Thus, the program is executed successfully...

Question 5
Correct
Marked out of 1.00
Flag question

To print unsigned values on the console, use %u format character instead of %d in the **printf()** function.

Whenever an attempt is made to assign a negative number to an **unsigned int** (For eg: unsigned int num = -1;) the compiler does not flag it as an **error**. Instead, it will automatically convert the negative number to a positive number as shown below:

unsigned int num = -1;
The value stored in num = unsigned int: maximum_value + 1 - num;
The final value in num = 4294967295 (in a 32-bit processing system)

In the program given below, fill in the missing **format characters** to print **signed** and **unsigned** values.

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     signed int number1 = -20, number2 = 20;
6     unsigned int number3 = -1, number4 = 1;
7     printf("Given signed values are %d and %d\n", number1, number2); // Fill the correct format character after %
8     printf("Given unsigned values are %u and %u\n", number3, number4); // Fill the correct format character after %
9     return 0;
10 }
```

	Expected	Got	
✓	Given signed values are -20 and 20	Given signed values are -20 and 20	✓
	Given unsigned values are 4294967295 and 1	Given unsigned values are 4294967295 and 1	

Passed all tests! ✓

➔ Thus, the program is executed successfully...

Question 6
Correct
Marked out of 1.00
Flag question

To print unsigned values on the console, use %u format character instead of %d in the **printf()** function.

Whenever an attempt is made to assign a negative number to an **unsigned int** (For eg: unsigned int num = -1;) the compiler does not flag it as an **error**. Instead, it will automatically convert the negative number to a positive number as shown below:

unsigned int num = -1;
The value stored in num = unsigned int: maximum_value + 1 - num;
The final value in num = 4294967295 (in a 32-bit processing system)

In the program given below, fill in the missing **format characters** to print **signed** and **unsigned** values.

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     signed int number1 = -20, number2 = 20;
6     unsigned int number3 = -1, number4 = 1;
7     printf("Given signed values are %d and %d\n", number1, number2); // Fill the correct format character after %
8     printf("Given unsigned values are %u and %u\n", number3, number4); // Fill the correct format character after %
9     return 0;
10 }
```

	Expected	Got	
✓	Given signed values are -20 and 20	Given signed values are -20 and 20	✓
	Given unsigned values are 4294967295 and 1	Given unsigned values are 4294967295 and 1	

Passed all tests! ✓

➔ Thus, the program is executed successfully...

Question 7
Correct
Marked out of 1.00
Flag question

To print unsigned values on the console, use %u format character instead of %d in the **printf()** function.

Whenever an attempt is made to assign a negative number to an **unsigned int** (For eg: unsigned int num = -1;) the compiler does not flag it as an **error**. Instead, it will automatically convert the negative number to a positive number as shown below:

unsigned int num = -1;
The value stored in num = unsigned int maximum_value + 1 - num;
The final value in num = 4294967295 (in a 32-bit processing system)

In the program given below, fill in the missing **format characters** to print **signed** and **unsigned** values.

Answer: (penalty regime: 0 %)

Reset answer

```

1 #include <stdio.h>
2
3 int main()
4 {
5     signed int number1 = -20, number2 = 20;
6     unsigned int number3 = -1, number4 = 1;
7     printf("Given signed values are %d and %d\n", number1, number2); // Fill the correct format character after %
8     printf("Given unsigned values are %u and %u\n", number3, number4); // Fill the correct format character after %
9     return 0;
10 }

```

Expected	Got
Given signed values are -20 and 20	Given signed values are -20 and 20
Given unsigned values are 4294967295 and 1	Given unsigned values are 4294967295 and 1

Passed all tests! ✓

➔ Thus, the program is executed successfully...

REC-Digital Cafe

CS23232-Fundamentals of Data Structures Using C-2025 Batch

Started on: Wednesday, 25 February 2026, 8:03 AM
State: Finished
Completed on: Wednesday, 25 February 2026, 8:21 AM
Time taken: 18 mins 43 secs
Name: SAKTHIVEL C S 2025-BME

Question 1
Correct
Marked out of 1.00
Flag question

Identify and correct the errors in the code given below:

Answer: (penalty regime: 0 %)

Reset answer

```

1 #include <stdio.h>
2
3 int main()
4 {
5     float num1 = 5.345f, num2 = 12.4, result;
6     printf("Given float values are num1 = %f, num2 = %f\n", num1, num2);
7     result = num1 / num2;
8     printf("Result of division = %f\n", result);
9     return 0;
10 }

```

Expected	Got
Given float values are num1 = 5.345000, num2 = 12.400000	Given float values are num1 = 5.345000, num2 = 12.400000
Result of division = 0.431048	Result of division = 0.431048

Passed all tests! ✓

➔ Thus, the program is executed successfully...

Question 2
Correct
Marked out of
1.00
Flag question

Identify and correct the errors in the code given below:

Expected Output:
Given float values are num1 = 5.340000, num2 = 125.789001
The result after dividing in float format = 23.555992
The result after dividing in exponential format = 2.355599e+01

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     float num1 = 5.34, num2 = 125.789f, result;
6     printf("Given float values are num1 = %f, num2 = %f\n", num1, num2);
7     result = num2 / num1;
8     printf("The result after dividing in float format = %f\n", result);
9     printf("The result after dividing in exponential format = %e\n", result);
10    return 0;
11 }
```

Expected	Got
✓ Given float values are num1 = 5.340000, num2 = 125.789001 The result after dividing in float format = 23.555992 The result after dividing in exponential format = 2.355599e+01	Given float values are num1 = 5.340000, num2 = 125.789001 The result after dividing in float format = 23.555992 The result after dividing in exponential format = 2.355599e+01 ✓

Passed all tests! ✓

➔ Thus, the program is executed successfully...

Question 1
Correct
Marked out of
1.00
Flag question

Arithmetic operators can be used in expressions with operands of different data types.

For example, the **addition operator (+)** can be used on **int** and **float** data types with the result being a **float** data type.

If the same is performed on **float** and **double** data types, the result will be a **double** data type, i.e., the result will always be of the **data type** with the highest memory capacity.

Consider the following examples with different combinations of arithmetic operators and data types.

Expression	Result	Description
2 + 2.5	4.500000	int + float = float
'A' + 10	75	char + int = int
5.9 + 4.67L	10.570000	float + long double = long double

Fill in the code given below with the correct missing **format characters**:

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("Result1 = %f\n", (2 + 2.5));
6     printf("Result2 = %d\n", ('A' + 10));
7     printf("Result3 = %Lf\n", (5.9 + 4.67L));
8     return 0;
9 }
```

Expected	Got
✓ Result1 = 4.500000 Result2 = 75 Result3 = 10.570000	Result1 = 4.500000 Result2 = 75 Result3 = 10.570000 ✓

Passed all tests! ✓

➔ Thus, the program is executed successfully...

Question 2

Correct

Marked out of 1.00

Flag question

The table given below demonstrates the use of various **arithmetic operators** using two variables c1 and c2 of type char with values 'A' and 'D' respectively:

Expression	Result
c1	65
c1 + c2	133
c1 + c2 + 5	138
c1 + c2 + 'S'	186

In the above examples, the character 'A' is substituted with its ASCII value 65 and 'D' is substituted with 68. The character 'S' is substituted with its ASCII value 83. The integer value 5 is used as it is.

The following table demonstrates the usage of various **arithmetic operators** using two variables a and b of type int with values 11 and -3 respectively:

Expression	Result
a + b	8
a - b	14
a * b	-33
a / b	-3
a % b	2

In the program given below, type the missing code to find the **result** of applying different **arithmetic operators** on **char** data type values.

Answer: (penalty regime: 0 %)

Reset answer

```

1 #include <stdio.h>
2
3 int main()
4 {
5     char c1 = 'A', c2 = 'D';
6     printf("c1 = %d\n", c1);
7     printf("c1 + c2 = %d\n", (c1 + c2));
8     printf("c1 + c2 + 5 = %d\n", (c1 + c2 + 5));
9     printf("Result = %d", (c1 + c2 + 'S'));
10    return 0;
11 }

```

	Expected	Got	
✓	c1 = 65	c1 = 65	✓
	c1 + c2 = 133	c1 + c2 = 133	
	c1 + c2 + 5 = 138	c1 + c2 + 5 = 138	
	Result = 186	Result = 186	

Passed all tests! ✓

➔ Thus, the program is executed successfully...

Question 3

Correct

Marked out of 1.00

Flag question

Division of one integer by another integer is referred to as **integer division**. This operation always results in an integer with truncated quotient.

If a **division** operation is carried out with two **floating point numbers** or with one **floating point number** and one **integer**, the result will be a **floating point quotient**.

The table given below demonstrates the usage of various **arithmetic operators** using two variables num1 and num2 of type float with values 12.5 and 2.0 respectively:

Expression	Result
num1 + num2	14.500000
num1 - num2	10.500000
num1 * num2	25.000000
num1 / num2	6.250000

Note that the **remainder operator (%)** is not applicable for **floating point numbers**.

In the program given below, type the missing code to find the **result** of applying different **arithmetic operators** on **floating point numbers**.

Answer: (penalty regime: 0 %)

Reset answer

```

1 #include <stdio.h>
2
3 int main()
4 {
5     float num1 = 12.5, num2 = 2.0;
6     printf("Result of addition = %f\n", (num1 + num2));
7     printf("Result of subtraction = %f\n", (num1 - num2));
8     printf("Result of multiplication = %f\n", (num1 * num2));
9     printf("Result of division = %f\n", (num1 / num2));
10    return 0;
11 }

```

	Expected	Got	
✓	Result of addition = 14.500000	Result of addition = 14.500000	✓
	Result of subtraction = 10.500000	Result of subtraction = 10.500000	
	Result of multiplication = 25.000000	Result of multiplication = 25.000000	
	Result of division = 6.250000	Result of division = 6.250000	

Passed all tests! ✓

➔ Thus, the program is executed successfully...

Lesson 4

Correct

Started out of 100

0% This question

An operator is a special symbol used to **manipulate data**. The data items that the operators act upon are called operands.

The operator that works on a single operand is called a **unary operator** and that which works on two operands is known as a **binary operator**.

C provides many types of operators. They are: Arithmetic, Logical, Relational and equality, Logical, Assignment, Conditional, Bitwise and Special operators.

In C, we have 5 arithmetic operators:

Operator Description

- + Used for addition
- Used for subtraction
- * Used for multiplication
- / Used for division
- % Remainder/Modulus operator for finding remainder

Arithmetic operators are applied on **numeric operands**. Thus the operands can be **integers, floats or characters** (Since a character is internally represented by its numeric code).

The **remainder operator (%)** requires that both the operands be **integers** and the second operand be **non-zero**. Similarly the **division operator (/)** requires that the second operand be **non-zero**.

The format for usage of arithmetic operator is as follows:

```
operand1operatoroperand2
```

According to the coding conventions in C, a single space should be provided to the left and to the right of an operator.

The table given below demonstrates the use of various **arithmetic operators** using two variables num1 and num2 of type int with values 10 and 3 respectively.

Expression Result

```
num1 + num2 13
num1 - num2 7
num1 * num2 30
num1 / num2 3
num1 % num2 1
```

Read the code given below to understand the usage of **arithmetic operators**. Fill in the space provided.

File: code.c

```
int main()
{
    int num1 = 10; num2 = 3;
    printf("Addition Result = %d\n", (num1 + num2));
    printf("Subtraction Result = %d\n", (num1 - num2));
    printf("Multiplication Result = %d\n", (num1 * num2));
    printf("Division Result = %d\n", (num1 / num2));
    printf("Remainder = %d\n", (num1 % num2));
    return 0;
}
```

Answer: (penalty regime: 0%)

```
1 //File:code.c:code.c:1
2 int main()
3 {
4     int num1 = 10; num2 = 3;
5     printf("Addition Result = %d\n", (num1 + num2));
6     printf("Subtraction Result = %d\n", (num1 - num2));
7     printf("Multiplication Result = %d\n", (num1 * num2));
8     printf("Division Result = %d\n", (num1 / num2));
9     printf("Remainder = %d\n", (num1 % num2));
10    return 0;
11 }
```

Expected	Got
✓ Addition Result = 13	Addition Result = 13 ✓
✓ Subtraction Result = 7	Subtraction Result = 7 ✓
✓ Multiplication Result = 30	Multiplication Result = 30 ✓
✓ Division Result = 3	Division Result = 3 ✓
✓ Remainder = 1	Remainder = 1 ✓

Passed all tests ✓

➔ Thus, the program is executed successfully...

REC Digital C++

Started on: Wednesday, 25 February 2020, 8:46 AM

State: Finished

Completed on: Wednesday, 25 February 2020, 8:56 AM

Time taken: 9 mins 10 secs

Name: SAKTHIARAJ, C.S.2020-BME

Relationship and equality operators are used to **test or compare** two numeric values or numeric expressions.

In C, **Relational and equality operators** when applied on the operands, produce an **integer** value which is either 0 or 1 and these are often referred to as logical values. The value 0 represents false and the value 1 represents true.

In C, there are **four relational and two equality operators** as given below:

Operator Description

- > Checks for greater than condition
- >= Checks for greater than or equals condition
- < Checks for less than condition
- <= Checks for less than or equals condition
- == Checks for two values are equal
- != Checks if two values are unequal

The format for usage of **relational and equality operators** is as follows:

```
operand1operatoroperand2
```

According to the coding conventions in C, a single space should be provided to the left and to the right of the operator.

The table given below demonstrates the use of various **relational and equality operators** using variables int num1 = 7, float num2 = 5.5, char ch = 'a'.

Expression Interpretation/Result Value

```
(num1 > 5) true 1
(num1 + num2) <= 10 false 0
(ch == 'A') true 1
(ch != 'a') true 1
(ch == 'A') || (num1 + num2) <= 10 false 0
```

Read the code given below and fill in the space provided.

File: code.c

```
int main()
{
    int num1 = 7;
    float num2 = 5.5;
    char ch = 'a';
    printf("Result1 = %d\n", (num1 > 5));
    printf("Result2 = %d\n", (num1 + num2) <= 10);
    printf("Result3 = %d\n", (ch == 'A'));
    printf("Result4 = %d\n", (ch != 'a'));
    printf("Result5 = %d\n", ((ch == 'A') || (num1 + num2) <= 10));
    return 0;
}
```

Answer: (penalty regime: 0%)

```
1 //File:code.c:code.c:1
2 int main()
3 {
4     int num1 = 7;
5     float num2 = 5.5;
6     char ch = 'a';
7     printf("Result1 = %d\n", (num1 > 5));
8     printf("Result2 = %d\n", (num1 + num2) <= 10);
9     printf("Result3 = %d\n", (ch == 'A'));
10    printf("Result4 = %d\n", (ch != 'a'));
11    printf("Result5 = %d\n", ((ch == 'A') || (num1 + num2) <= 10));
12    return 0;
13 }
```

Expected	Got
✓ Result1 = 1	Result1 = 1 ✓
✓ Result2 = 0	Result2 = 0 ✓
✓ Result3 = 0	Result3 = 0 ✓
✓ Result4 = 1	Result4 = 1 ✓
✓ Result5 = 1	Result5 = 1 ✓

Passed all tests ✓

➔ Thus, the program is executed successfully...

CS23232-Fundamentals of Data Structures Using C-2025 Batch

Started on: Wednesday, 25 February 2026, 8:58 AM
 State: Finished
 Completed on: Wednesday, 25 February 2026, 9:05 AM
 Time taken: 7 mins 37 secs
 Name: SAKTHIHEL, C.S.2025 BMS

Logical operators are used to perform logical operations on the given expressions.
 An expression containing a logical operator returns either 0 (or) 1, depending on the evaluation of the expression to either false or true respectively.
Note: In C, false is represented as 0 (zero) and all non-zero values can be treated as true.

Given below are the three logical operators in C:

Operator Description Meaning
 && logical AND: It returns true when both conditions are true, else it returns false.
 || logical OR: It returns true if atleast one of the conditions is true.
 ! logical NOT: It returns true when the given expression is false and returns false when the given expression is true.

According to the coding conventions in C, a single space should be provided to the left and to the right of the operator.

The below table demonstrates the use of various relational and equality operators using variable int num1 = 5, float num2 = 5.5, char ch = 'a'.

Expression	Interpretation/Result Value
num1 >= 6 && ch == 'a'	true 1
num1 < 7.5 && num2 > 100	false 0
ch != 'p' (num1 > num2) <= 10	true 1
num1 > num2 < 10	false 0
num1 <= 5	true 1

Read the code given below and replace in the space provided.

```
#include <stdio.h>

int main()
{
    int num1 = 5;
    float num2 = 5.5;
    char ch = 'a';
    printf("num1 <= num2? (num1 = %d, num2 = %f) : ", num1, num2);
    printf("num2 <= num1? (num2 = %f, num1 = %d) : ", num2, num1);
    printf("num1 < num2? (num1 = %d, num2 = %f) : ", num1, num2);
    printf("num2 < num1? (num2 = %f, num1 = %d) : ", num2, num1);
    printf("num1 <= num2? (num1 = %d, num2 = %f) : ", num1, num2);
    printf("num2 <= num1? (num2 = %f, num1 = %d) : ", num2, num1);
    return 0;
}
```

Answer: (formatly replace 0/0)

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int num1 = 5;
6     float num2 = 5.5;
7     char ch = 'a';
8     printf("num1 <= num2? (num1 = %d, num2 = %f) : ", num1, num2);
9     printf("num2 <= num1? (num2 = %f, num1 = %d) : ", num2, num1);
10    printf("num1 < num2? (num1 = %d, num2 = %f) : ", num1, num2);
11    printf("num2 < num1? (num2 = %f, num1 = %d) : ", num2, num1);
12    printf("num1 <= num2? (num1 = %d, num2 = %f) : ", num1, num2);
13    printf("num2 <= num1? (num2 = %f, num1 = %d) : ", num2, num1);
14    return 0;
15 }
```

Expected	Got
num1 <= num2? (num1 = 5, num2 = 5.5) : 0	num1 <= num2? (num1 = 5, num2 = 5.5) : 0
num2 <= num1? (num2 = 5.5, num1 = 5) : 0	num2 <= num1? (num2 = 5.5, num1 = 5) : 0
num1 < num2? (num1 = 5, num2 = 5.5) : 0	num1 < num2? (num1 = 5, num2 = 5.5) : 0
num2 < num1? (num2 = 5.5, num1 = 5) : 0	num2 < num1? (num2 = 5.5, num1 = 5) : 0

Passed all tests! ✓

➔ Thus, the program is executed successfully...

CS23232-Fundamentals of Data Structures Using C-2025 Batch

Started on: Wednesday, 25 February 2026, 9:12 AM
 State: Finished
 Completed on: Wednesday, 25 February 2026, 9:31 AM
 Time taken: 19 mins 20 secs
 Name: SAKTHIHEL, C.S.2025 BMS

Read the code given below to understand the working of unary operators. Replace in the space provided.

```
#include <stdio.h>

int main()
{
    int x = 10;
    printf("x = %d", x);
    printf("x = %d", ++x);
    printf("x = %d", x);
    printf("x = %d", --x);
    printf("x = %d", x);
    printf("x = %d", ++x);
    printf("x = %d", x);
    printf("x = %d", --x);
    printf("x = %d", x);
    printf("x = %d", ++x);
    printf("x = %d", x);
    printf("x = %d", --x);
    printf("x = %d", x);
    return 0;
}
```

Answer: (formatly replace 0/0)

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int x = 10;
6     printf("x = %d", x);
7     printf("x = %d", ++x);
8     printf("x = %d", x);
9     printf("x = %d", --x);
10    printf("x = %d", x);
11    printf("x = %d", ++x);
12    printf("x = %d", x);
13    printf("x = %d", --x);
14    printf("x = %d", x);
15    printf("x = %d", ++x);
16    printf("x = %d", x);
17    printf("x = %d", --x);
18    printf("x = %d", x);
19    return 0;
20 }
```

Expected	Got
x = 10	x = 10
x = 11	x = 11
x = 10	x = 10
x = 9	x = 9
x = 10	x = 10
x = 11	x = 11
x = 10	x = 10
x = 9	x = 9
x = 10	x = 10
x = 11	x = 11
x = 10	x = 10
x = 9	x = 9
x = 10	x = 10

Passed all tests! ✓

➔ Thus, the program is executed successfully...

Question 2
Correct
Marked out of 1.00
Flag question

Read the code given below to understand the working of **increment** and **decrement** operators. Retype in the space provided.

```
#include <stdio.h>

int main()
{
    int x = 4, y;
    y = x++;
    printf("y = %d x = %d\n", y, x);
    y = ++x;
    printf("y = %d x = %d\n", y, x);
    y = x--;
    printf("y = %d x = %d\n", y, x);
    y = --x;
    printf("y = %d x = %d\n", y, x);
    return 0;
}
```

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int x = 4, y;
6     y = x++;
7     printf("y = %d x = %d\n", y, x);
8     y = ++x;
9     printf("y = %d x = %d\n", y, x);
10    y = x--;
11    printf("y = %d x = %d\n", y, x);
12    y = --x;
13    printf("y = %d x = %d\n", y, x);
14    return 0;
15 }
```

Expected	Got	
y = 5 x = 5	y = 5 x = 5	✓
y = 6 x = 6	y = 6 x = 6	✓
y = 5 x = 5	y = 5 x = 5	✓
y = 4 x = 4	y = 4 x = 4	✓

Passed all tests! ✓

➔ Thus, the program is executed successfully...

Question 1
Correct
Marked out of 1.00
Flag question

Read the code given below to understand the usage of the assignment operator. Retype in the space provided.

```
#include <stdio.h>

int main()
{
    int x = 24, y = 39, z = 45;
    z = x + y;
    y = z - y;
    x = z - y;
    printf("x = %d y = %d z = %d", x, y, z);
    return 0;
}
```

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int x = 24, y = 39, z = 45;
6     z = x + y;
7     y = z - y;
8     x = z - y;
9     printf("x = %d y = %d z = %d", x, y, z);
10    return 0;
11 }
```

Expected	Got	
x = 39 y = 24 z = 63	x = 39 y = 24 z = 63	✓

Passed all tests! ✓

➔ Thus, the program is executed successfully...

Question 2
Correct
Marked out of 1.00
Flag question

Read the code given below and retype in the space provided.

```
#include <stdio.h>

int main()
{
    int x = 2, y = 18, z = 12;
    x += y;
    printf("x = %d\n", x);
    y /= 2;
    printf("y = %d\n", y);
    z /= 5;
    printf("z = %d\n", z);
    x %= 7;
    printf("x = %d", x);
    return 0;
}
```

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int x = 2, y = 18, z = 12;
6     x += y;
7     printf("x = %d\n", x);
8     y /= 2;
9     printf("y = %d\n", y);
10    z /= 5;
11    printf("z = %d\n", z);
12    x %= 7;
13    printf("x = %d", x);
14    return 0;
15 }
```

Expected	Got
✓ x = 20 y = 36 z = 2 x = 6	x = 20 y = 36 z = 2 x = 6 ✓

Passed all tests! ✓

➔ Thus, the program is executed successfully...

Question 1
Correct
Marked out of 1.00
Flag question

C language provides an operator to evaluate conditions. It is called a **conditional operator** (?) or a **ternary operator**.

Ternary operator needs exactly **three** operands to compute the result.

The syntax for using a **ternary operator** is:

condition? expression1 : expression2

The condition should always evaluate to 1 (true) or 0 (false). If the condition evaluates to true, then **expression1** is evaluated and its value is returned, otherwise **expression2** is evaluated and its value is returned.

Given below is an example demonstrating the usage of a **conditional operator**:

```
int marks = 75, pass_marks = 50;
(marks > pass_marks) ? printf("Passed C Certification") : printf("Failed C Certification");
```

Since the condition marks > pass_marks evaluates to true, the **expression1** containing the **printf** statement in the above example prints "Passed C Certification"

Read the code given below and retype in the space provided.

```
#include <stdio.h>

int main()
{
    int marks = 75, pass_marks = 50;
    (marks > pass_marks) ? printf("Passed C exam.") : printf("Failed C exam.");
    return 0;
}
```

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int marks = 75, pass_marks = 50;
6     (marks > pass_marks) ? printf("Passed C exam.") : printf("Failed C exam.");
7     return 0;
8 }
9 }
```

Expected	Got
✓ Passed C exam.	Passed C exam. ✓

Passed all tests! ✓

➔ Thus, the program is executed successfully...

Question 2
Correct
Marked out of
1.00
Flag question

In the program given below, fill in the missing code to find the **largest** of the two given numbers using **ternary operator**.

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int num1 = 20, num2 = 25, large;
6     large = (num1 > num2) ? num1 : num2; // Write the correct code
7     printf("Largest number = %d", large);
8     return 0;
9 }
```

Expected	Got
✓ Largest number = 25	Largest number = 25 ✓

Passed all tests! ✓

➔ Thus, the program is executed successfully...

Question 1
Correct
Marked out of
1.00
Flag question

There was a large ground in center of the city which is rectangular in shape. The Corporation decides to build a Cricket stadium in the area for school and college students. But the area was used as a car parking zone. In order to protect the land from using as an unauthorized parking zone, the corporation wanted to protect the stadium by building a fence. In order to help the workers to build a fence, they planned to place a thick rope around the ground. They wanted to buy only the exact length of the rope that is needed. They also wanted to cover the entire ground with a carpet during rainy season. They wanted to buy only the exact quantity of carpet that is needed. They requested your help. Can you please help them by writing a program to find the exact length of the rope and the exact quantity of carpet that is required?

Input format:

Input consists of 2 integers. The first integer corresponds to the length of the ground and the second integer corresponds to the breadth of the ground.

Output Format:

Output Consists of two integers. The first integer corresponds to the length. The second integer corresponds to the quantity of carpet required.

Sample Input:

50

20

Sample Output:

140

1000

For example:

Input	Result
50	140
20	1000

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int length, breadth;
6     int Perimeter, Area;
7     scanf("%d\n", &length);
8     scanf("%d\n", &breadth);
9     Perimeter = 2 * (length + breadth);
10
11     Area = length * breadth;
12
13     printf("%d\n", Perimeter);
14     printf("%d\n", Area);
15     return 0;
16 }
```

	Input	Expected	Got
✓	50	140	140 ✓
	20	1000	1000 ✓

Passed all tests! ✓

➔ Thus, the program is executed successfully...

Question 1
Correct
Marked out of 1.00
1 Flag question

Training for sports day has begun and the physical education teacher has decided to conduct some team games. The teacher wants to split the students in higher secondary into equal sized teams. In some cases, there may be some students who are left out from the teams and he wanted to use the left out students to assist him in conducting the team games. For instance, if there are 50 students in a class and if the class has to be divided into 7 equal sized teams, 7 students will be there in each team and 1 student will be left out. That 1 student will assist the PET. With this idea in mind, the PET wants your help to automate this team splitting task. Can you please help him out?

INPUT FORMAT:

Input consists of 2 integers. The first integer corresponds to the number of students in the class and the second integer corresponds to the number of teams.

OUTPUT FORMAT:

The output consists of two integers. The first integer corresponds to the number of students in each team and the second integer corresponds to the students who are left out.

SAMPLE INPUT:

60

8

SAMPLE OUTPUT:

7

4

For example:

Input	Result
60	7
8	4

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int total_students, num_teams;
6     int students_per_team, left_out;
7
8     if (scanf("%d %d", &total_students, &num_teams) == 2);
9     {
10         students_per_team = total_students / num_teams;
11         left_out = total_students % num_teams;
12
13         printf("%d\n", students_per_team);
14         printf("%d\n", left_out);
15     }
16     return 0;
17 }
```

Input	Expected	Got
✓ 60	7	7
8	4	4 ✓

Passed all tests! ✓

➔ Thus, the program is executed successfully...

Question 1
Correct
Marked out of 1.00
1 Flag question

name : SARATHI VELU S J 2023-BME

Each Sunday, a newspaper agency sells w copies of a special edition newspaper for Rs. x per copy. The cost to the agency of each newspaper is Rs. y . The agency pays a fixed cost for storage, delivery and so on of Rs.100 per Sunday. The newspaper agency wants to calculate the profit which it obtains only on Sundays. Can you please help them out by writing a program to compute the profit if w , x , and y are given?

INPUT FORMAT:

Input consists of 3 integers: w , x , and y . w is the number of copies sold, x is the cost per copy and y is the cost the agency spends per copy.

OUTPUT FORMAT:

The output consists of a single integer which corresponds to the profit obtained by the newspaper agency.

SAMPLE INPUT:

1000

2

1

SAMPLE OUTPUT:

900

For example:

Input	Result
1000	900
2	
1	

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int w, x, y;
6     int profit;
7
8     scanf("%d\n", &w);
9     scanf("%d\n", &x);
10    scanf("%d\n", &y);
11
12    profit = (w * x) - (w * y) - 100;
13
14    printf("%d\n", profit);
15    return 0;
16 }
```

Input	Expected	Got
✓ 1000	900	900 ✓
2		
1		

Passed all tests! ✓

➔ Thus, the program is executed successfully...

REC-Digital Cafe

Completed on

Sunday, 1 March 2026, 9:51 AM

Time taken

5 mins 10 secs

Name

SAKTHIVEL C S 2025-BME

Question 1

Correct

Marked out of 1.00

Flag question

Four kids Peter, Susan, Edmond and Lucy travel through a wardrobe to the land of Narnia. Narnia is a fantasy world of magic with mythical beasts and talking animals. While exploring the land of narnia Lucy found Mr.Tumnus the two legged stag, and she followed it, down a narrow path. She and Mr.Tumnus became friends and he offered a cup of coffee to Lucy in his small hut. It was time for Lucy to return to her family and so she bid good bye to Mr.Tumnus and while leaving Mr.Tumnus told that it is quite difficult to find the route back as it was already dark. He told her to see the trees while returning back and said that the first tree with two digits number will help her find the way and the way to go back to her home is the sum of digits of the tree and that numbered way will lead her to the tree next to the wardrobe where she can find the others. Lucy was already confused, so please help her in finding the route to her home...

Input Format:

Input consists of an integer corresponding to the 2-digit number.

Output Format:

Output consists of an integer corresponding to the sum of its digits.

SAMPLE INPUT:

87

SAMPLE OUTPUT:

15

For example:

Input	Result
87	15

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int number, tens, ones, sum;
6     scanf("%d\n", &number);
7
8     tens = number / 10;
9     ones = number % 10;
10
11     sum = tens + ones;
12     printf("%d\n", sum);
13
14     return 0;
15 }
16
17 }
```

	Input	Expected	Got	
✓	87	15	15	✓
✓	54	9	9	✓

Passed all tests! ✓

➔ Thus, the program is executed successfully...